FAILURE-AWARE EPISODIC MEMORY FOR PENETRATION-TESTING AGENTS

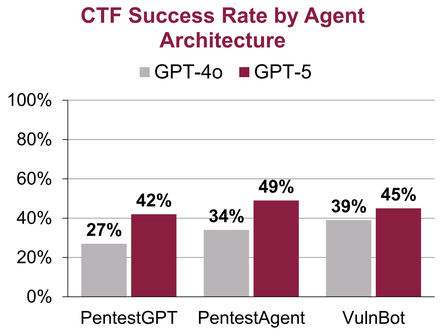
Shrikant Bhatnagar; Dr. Keke Chen, Ph.D. (Email: vt06290@umbc.edu)
Department of Computer Science and Engineering

Trustworthy and Intelligent Computing Lab

UMBC

Research problem

- Penetration testing, a practice of probing systems for exploitable weaknesses before real attackers do, is essential to securing software, but it is labour-intensive and requires scarce expertise: ISC2 estimates a global shortfall of roughly 4.7 million cybersecurity professionals.
- Large language models can now attempt penetration testing autonomously, promising to reduce assessment cost and make advanced testing accessible to organizations that can't afford dedicated expertise.



- Current agents, however, remain immature. On benchmarks they solve only easy and moderate difficulty challenges.
- Existing architectures treat failure as something to reflect on in the moment rather than knowledge to be retained and reused.

This project asks:

Can failure-aware structured episodic memory reduce repeated ineffective exploration and improve the performance of autonomous penetration-testing agents?

Background

LLM Agents:

An LLM becomes an agent when it is given tools (shell access, APIs etc) and a scaffold that loops: propose a command, run it, read the output, decide the next move. Unlike a chatbot, which answers once and waits, an agent executes against the live target and adapts over dozens of steps without a human in the loop.

Penetration-Testing:

Penetration testing is the practice of simulating an attack against a system, with authorization, in order to find vulnerabilities before a real attacker does.

A typical penetration testing task follows a loose cycle:

1. Reconnaissance Map the target to find open ports and exposed files → **2. Enumeration** Probe those services for specific weaknesses → **3. Exploitation** Use a weakness to gain access or extract information

repeat as new services and weaknesses surface

Capture-the-Flag:

A Capture-the-Flag (CTF) challenges is a deliberately vulnerable system containing a hidden string called a flag. Recovering the flag is proof that the system was compromised, which makes CTFs one of the few security tasks with an automatically checkable success signal. This project uses challenges drawn from the **Cybench** benchmark suite.

Methodology

Baseline Selection:

PentestGPT was selected as the baseline because:

- **Most established baseline:** published at USENIX Security, widely cited as the reference point for later LLM pentesting agents.
- **Modular by design:** separates reasoning, generation, and parsing, making it a clean substrate to instrument and extend.
- **Tracks tasks, not failures:** its Pentesting Task Tree records task structure but retains no memory of failed techniques. That gap is exactly what FAEM targets.

Failure Classification:

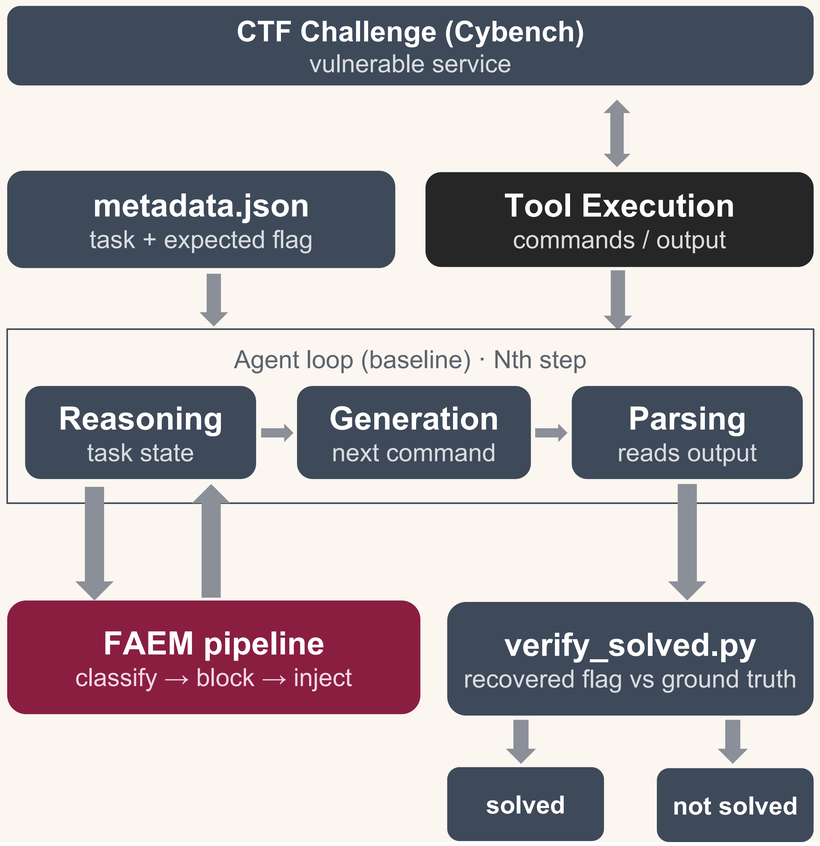
Each failed turn is classified into of two types.
Following is the failure taxonomy:

- **Type A (capability gaps):** wrong tool or syntax, misread output. Shrink as models improve.
- **Type B (complexity barriers):** forgetting findings, premature commitment, unproductive exploration. Persist regardless of model.

Classification turns a failure into a decision: retry with a fix (Type A) or abandon the approach as a dead-end (Type B).

This taxonomy was followed from the "What Makes a Good LLM Agent for Penetration Testing (2026 preprint)" work.

Comparison Framework



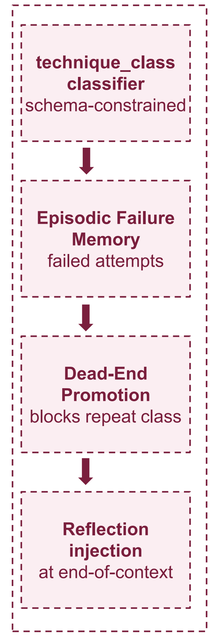
Contributions

Failure-Aware Episodic Memory:

- **Episodic memory** lets an agent store previous actions, observations, reasoning traces, and outcomes, and retrieve them to make better future decisions.
- **Failure-Aware Episodic Memory (FAEM)** stores failed attempts, dead-ends, and unsuccessful reasoning as structured, retrievable memory, so the agent stops re-trying what already failed.

FAEM Pipeline:

- **Classify:** each failure is tagged with a technique class (schema-constrained, so tagging is reliable), capturing what kind of approach failed, not its exact syntax.
- **Store:** the tagged failure enters episodic failure memory.
- **Promote:** when a class fails repeatedly, it's promoted to a dead-end and blocked from re-selection.
- **Inject:** the failure record is fed back into context where the agent plans its next move.



Conclusion and Future Work

- Autonomous penetration-testing agents waste much of their limited turn budget re-attempting approaches that have already failed.
- To address this, we reimplemented and instrumented a PentestGPT baseline and built a failure-aware episodic memory that classifies failures by technique, promotes repeat offenders to dead-ends, and blocks the agent from re-trying them.
- Baseline runs confirm the failure mode is real and pervasive, establishing the harness and the problem characterization needed to evaluate whether structured failure memory can reduce ineffective exploration.
- Collect failed trajectories from one challenge set, feed them to FAEM, and test whether they improve performance on a second, unseen set. Because the memory holds only failures, never solutions, any gain measures transfer of what-doesn't-work.

Acknowledgments

This research was conducted in the Trustworthy and Intelligent Computing lab under the COEIT Summer Student Program (CSSP).